МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №6

з дисципліни
«Об’єктно-орієнтоване програмування»

на тему
«Побудування програмної системи з множини об’єктів, керованих повідомленнями»

Виконав:
студент групи ІМ-55
Стеблевець Олександр Олександрович
Номер в списку – 26

Перевірив:
Рекечинський Д.О.

Київ 2026

Мета роботи

Отримати вміння та навички використовувати засоби обміну інформацією та запрограмувати взаємодію незалежно працюючих програмних компонентів.

Завдання та варіант

Номер у журналі – 26, тому номер варіанта завдання: 26 mod 4 = 2. Необхідно створити три незалежні програми: Lab6, Object2 та Object3. Програма Lab6 приймає параметри n, Min і Max та керує іншими програмами. Object2 генерує вектор із n дробових чисел у заданому діапазоні, відображає його та записує в текстовому форматі в буфер обміну. Object3 зчитує вектор із буфера, сортує його та відображає результат.

Програма Lab6 знаходить уже запущені об’єкти або запускає їх, забезпечує послідовний обмін повідомленнями та дозволяє повторно виконувати завдання. При завершенні Lab6 надсилає об’єктам команди завершення.

Лістинги програми

Обмін пакетами між програмами реалізовано через іменовані локальні сокети за допомогою бібліотеки interprocess. Спільна бібліотека common містить типи пакетів, засоби обміну повідомленнями та роботи з буфером обміну.

Lab6

```rust
// SPDX-FileCopyrightText: 2026 Oleksandr Steblevets
// SPDX-License-Identifier: GPL-3.0-only

fn main() {
    slint_build::compile("src/app.slint").unwrap();
}
```

Лістинг 1 – lab6/build.rs

```rust
// SPDX-FileCopyrightText: 2026 Oleksandr Steblevets
// SPDX-License-Identifier: GPL-3.0-only

mod manager;
mod ui;

use anyhow::Result;
use slint::ComponentHandle;
```

```
use tokio::sync::mpsc;

slint::include_modules!();

#[tokio::main]
async fn main() -> Result<()> {
    let app = App::new()?;
    let dialog = Parameters::new()?;
    let (commands, receiver) = mpsc::unbounded_channel();
    let manager = tokio::spawn(manager::run(receiver, app.as_weak()));

    ui::configure(&app, &dialog, commands.clone());
    let result = app.run();
    let _ = commands.send(manager::ManagerCommand::Shutdown);
    manager.await?;
    result?;
    Ok(())
}
```

Лістинг 2 – lab6/src/main.rs

```
// SPDX-FileCopyrightText: 2026 Oleksandr Steblevets
// SPDX-License-Identifier: GPL-3.0-only

use crate::App;
use anyhow::Result;
use common::{
    communicate::{self, Communicate},
    packets::{MANAGER, OBJECT2, OBJECT3, Packet},
};
use std::{io, time::Duration};
use tokio::{process::Command, sync::mpsc, time::Instant};

struct Manager {
    communicate: Communicate,
    next_id: u64,
}

impl Manager {
    fn next_id(&mut self) -> u64 {
        self.next_id += 1;
        self.next_id
    }

    async fn wait_for(&mut self, expected: u64) -> Result<()> {
        tokio::time::timeout(Duration::from_secs(10), async {
            loop {
                if let Packet::Finished { id, success } = self.communicate.receive_packet().await? {
                    if id == expected {
                        return if success { Ok(()) } else { Err(io::Error::other("")) };
                    }
                }
            }
        }).await??;
        Ok(())
    }

    async fn ensure_worker(&mut self, name: &'static str) -> Result<()> {
        let id = self.next_id();
        let ping = Packet::Ping { id };
        match self.communicate.send_packet(name, &ping).await {
            Ok(()) => return self.wait_for(id).await,
            Err(error) if matches!(error.kind(), io::ErrorKind::NotFound |
io::ErrorKind::ConnectionRefused) => {}
            Err(error) => return Err(error.into()),
        }

        let executable = std::env::current_exe()?
            .with_file_name(format!("{name}{}", std::env::consts::EXE_SUFFIX));
        let mut child = Command::new(executable).spawn()?;
        let deadline = Instant::now() + Duration::from_secs(10);
        while Instant::now() < deadline {
            if self.communicate.send_packet(name, &ping).await.is_ok() {
                return self.wait_for(id).await;
            }
            if child.try_wait()?.is_some() {
```

```
                return Err(io::Error::from(io::ErrorKind::ConnectionAborted).into());
            }
            tokio::time::sleep(Duration::from_millis(100)).await;
        }
        let _ = child.kill().await;
        Err(io::Error::from(io::ErrorKind::TimedOut).into())
    }

    async fn execute(&mut self, n: usize, min: f64, max: f64) -> Result<()> {
        self.ensure_worker(OBJECT2).await?;
        let id = self.next_id();
        self.communicate.send_packet(OBJECT2, &Packet::Generate { id, n, min, max }).await?;
        self.wait_for(id).await?;

        self.ensure_worker(OBJECT3).await?;
        let id = self.next_id();
        self.communicate.send_packet(OBJECT3, &Packet::Sort { id }).await?;
        self.wait_for(id).await
    }
}

pub enum ManagerCommand {
    Execute { n: usize, min: f64, max: f64 },
    Shutdown,
}

pub async fn run(mut commands: mpsc::UnboundedReceiver<ManagerCommand>, app: slint::Weak<App>) {
    let Ok(communicate) = communicate::create(MANAGER) else { return };
    let mut manager = Manager { communicate, next_id: 0 };
    while let Some(command) = commands.recv().await {
        match command {
            ManagerCommand::Execute { n, min, max } => {
                let result = manager.execute(n, min, max).await;
                let _ = app.upgrade_in_event_loop(move |app| {
                    app.set_busy(false);
                    app.set_status(if result.is_ok() {
                        "Готово. Можна виконати завдання повторно.".into()
                    } else {
                        "".into()
                    });
                });
            }
            ManagerCommand::Shutdown => break,
        }
    }
    for name in [OBJECT2, OBJECT3] {
        let _ = manager.communicate.send_packet(name, &Packet::Shutdown).await;
    }
}
```

Лістинг 3 – lab6/src/manager.rs

```
// SPDX-FileCopyrightText: 2026 Oleksandr Steblevets
// SPDX-License-Identifier: GPL-3.0-only

use crate::{App, Parameters, manager::ManagerCommand};
use common::validate;
use slint::{ComponentHandle, LogicalPosition};
use tokio::sync::mpsc;

pub fn configure(app: &App, dialog: &Parameters, commands: mpsc::UnboundedSender<ManagerCommand>) {
    app.window().on_close_requested(|| {
        let _ = slint::quit_event_loop();
        slint::CloseRequestResponse::HideWindow
    });
    app.window().set_position(LogicalPosition::new(20.0, 20.0));
    let dialog_weak = dialog.as_weak();
    app.on_open_parameters(move || {
        if let Some(dialog) = dialog_weak.upgrade() {
            let _ = dialog.show();
        }
    });

    let app_weak = app.as_weak();
    let dialog_weak = dialog.as_weak();
    dialog.on_execute(move |n, min, max| {
```

```
        let parsed = n
            .trim()
            .parse::<usize>()
            .ok()
            .zip(min.trim().parse::<f64>().ok())
            .zip(max.trim().parse::<f64>().ok())
            .map(|((n, min), max)| (n, min, max));
        let Some((n, min, max)) = parsed else {
            return;
        };
        if validate(min, max).is_err() {
            return;
        }
        if let Some(dialog) = dialog_weak.upgrade() {
            let _ = dialog.hide();
        }
        if let Some(app) = app_weak.upgrade() {
            app.set_busy(true);
            app.set_status("Виконання...".into());
        }
        if commands.send(ManagerCommand::Execute { n, min, max }).is_err() {
            if let Some(app) = app_weak.upgrade() {
                app.set_busy(false);
                app.set_status("".into());
            }
        }
    });
}
```

Лістинг 4 – lab6/src/ui.rs

```
// SPDX-FileCopyrightText: 2026 Oleksandr Steblevets
// SPDX-License-Identifier: GPL-3.0-only

import { Button, LineEdit, VerticalBox, HorizontalBox } from "std-widgets.slint";

export component App inherits Window {
    title: "Lab6";
    width: 420px;
    height: 180px;
    in-out property <string> status: "Оберіть завдання, щоб почати.";
    in-out property <bool> busy: false;
    callback open-parameters();
    MenuBar {
        Menu {
            title: "Завдання";
            MenuItem {
                title: "Сформувати та відсортувати вектор";
                enabled: !root.busy;
                activated => { root.open-parameters(); }
            }
        }
    }
    VerticalBox {
        padding: 16px;
        spacing: 12px;
        Text { text: root.status; wrap: word-wrap; }
    }
}

export component Parameters inherits Window {
    title: "Параметри вектора";
    width: 350px;
    height: 250px;
    callback execute(string, string, string);
    VerticalBox {
        padding: 16px;
        spacing: 8px;
        HorizontalBox { Text { text: "Кількість (n):"; width: 120px; } n-input := LineEdit { text:
"20"; } }
        HorizontalBox { Text { text: "Мінімум:"; width: 120px; } min-input := LineEdit { text:
"-10"; } }
        HorizontalBox { Text { text: "Максимум:"; width: 120px; } max-input := LineEdit { text:
"10"; } }
        Button { text: "Виконати"; clicked => { root.execute(n-input.text, min-input.text, max-
input.text); } }
```

```
    }
}
```

Лістинг 5 – lab6/src/app.slint

Object2

```
// SPDX-FileCopyrightText: 2026 Oleksandr Steblevets
// SPDX-License-Identifier: GPL-3.0-only

fn main() {
    slint_build::compile("src/app.slint").unwrap();
}
```

Лістинг 6 – object2/build.rs

```
// SPDX-FileCopyrightText: 2026 Oleksandr Steblevets
// SPDX-License-Identifier: GPL-3.0-only

mod ui;
mod worker;

use anyhow::Result;
use common::{communicate, packets::OBJECT2};
use slint::ComponentHandle;

slint::include_modules!();

#[tokio::main]
async fn main() -> Result<()> {
    let app = App::new()?;
    ui::configure(&app);
    let communicate = communicate::create(OBJECT2)?;
    let (stop, stopped) = tokio::sync::oneshot::channel();
    let task = tokio::spawn(worker::run(communicate, app.as_weak(), stopped));

    let result = app.run();
    let _ = stop.send(());
    task.await?;
    result?;
    Ok(())
}
```

Лістинг 7 – object2/src/main.rs

```
// SPDX-FileCopyrightText: 2026 Oleksandr Steblevets
// SPDX-License-Identifier: GPL-3.0-only

use crate::{App, ui};
use anyhow::Result;
use common::{
    clipboard::Clipboard,
    communicate::Communicate,
    packets::{MANAGER, Packet},
};
use rand::Rng;
use tokio::sync::oneshot;

pub async fn run(
    mut communicate: Communicate,
    app: slint::Weak<App>,
    mut stopped: oneshot::Receiver<()>,
) {
    let mut clipboard = Clipboard::default();
    loop {
        let packet = tokio::select! {
            packet = communicate.receive_packet() => match packet {
                Ok(packet) => packet,
                Err(_) => break,
            },
            _ = &mut stopped => return,
        };
        let (id, result) = match packet {
            Packet::Ping { id } => (id, Ok(())),
            Packet::Generate { id, n, min, max } => {
```

```
                let result: Result<()> = (|| {
                    let mut rng = rand::rng();
                    let values: Vec<f64> = (0..n).map(|_| {
                        if min == max { min } else { rng.random_range(min..=max) }
                    }).collect();
                    clipboard.write_vector(&values)?;
                    ui::show_values(&app, &values)
                })();
                (id, result)
            }
            Packet::Shutdown => break,
            _ => continue,
        };
        let _ = communicate.send_packet(MANAGER, &Packet::Finished { id, success:
result.is_ok() }).await;
    }
    ui::quit();
}
```

Лістинг 8 – object2/src/worker.rs

```
// SPDX-FileCopyrightText: 2026 Oleksandr Steblevets
// SPDX-License-Identifier: GPL-3.0-only

use crate::App;
use anyhow::Result;
use slint::{ComponentHandle, LogicalPosition};

pub fn configure(app: &App) {
    app.window().set_position(LogicalPosition::new(20.0, 240.0));
}

pub fn show_values(app: &slint::Weak<App>, values: &[f64]) -> Result<()> {
    let display = common::format_values(values);
    app.upgrade_in_event_loop(move |app| app.set_values(display.into()))
        .map_err(|error| anyhow::anyhow!(error.to_string()))?;
    Ok(())
}

pub fn quit() {
    let _ = slint::invoke_from_event_loop(|| { let _ = slint::quit_event_loop(); });
}
```

Лістинг 9 – object2/src/ui.rs

```
    // SPDX-FileCopyrightText: 2026 Oleksandr Steblevets
    // SPDX-License-Identifier: GPL-3.0-only

    import { ScrollView, VerticalBox } from "std-widgets.slint";
    export component App inherits Window {
        title: "Object2";
        width: 500px;
        height: 400px;
        in-out property <string> values: "Очікування даних від Lab6...";
        VerticalBox {
            padding: 12px;
            spacing: 8px;
            Text { text: "Згенерований вектор:"; }
            ScrollView { Text { text: root.values; font-family: "monospace"; } }
        }
    }
```

Лістинг 10 – object2/src/app.slint

Object3

```
// SPDX-FileCopyrightText: 2026 Oleksandr Steblevets
// SPDX-License-Identifier: GPL-3.0-only

fn main() {
    slint_build::compile("src/app.slint").unwrap();
}
```

Лістинг 11 – object3/build.rs

```rust
// SPDX-FileCopyrightText: 2026 Oleksandr Steblevets
// SPDX-License-Identifier: GPL-3.0-only

mod ui;
mod worker;

use anyhow::Result;
use common::{communicate, packets::OBJECT3};
use slint::ComponentHandle;

slint::include_modules!();

#[tokio::main]
async fn main() -> Result<()> {
    let app = App::new()?;
    ui::configure(&app);
    let communicate = communicate::create(OBJECT3)?;
    let (stop, stopped) = tokio::sync::oneshot::channel();
    let task = tokio::spawn(worker::run(communicate, app.as_weak(), stopped));

    let result = app.run();
    let _ = stop.send(());
    task.await?;
    result?;
    Ok(())
}
```

Лістинг 12 – object3/src/main.rs

```rust
// SPDX-FileCopyrightText: 2026 Oleksandr Steblevets
// SPDX-License-Identifier: GPL-3.0-only

use crate::{App, ui};
use anyhow::Result;
use common::{
    clipboard::Clipboard,
    communicate::Communicate,
    packets::{MANAGER, Packet},
};
use tokio::sync::oneshot;

pub async fn run(
    mut communicate: Communicate,
    app: slint::Weak<App>,
    mut stopped: oneshot::Receiver<()>,
) {
    let mut clipboard = Clipboard::default();
    loop {
        let packet = tokio::select! {
            packet = communicate.receive_packet() => match packet {
                Ok(packet) => packet,
                Err(_) => break,
            },
            _ = &mut stopped => return,
        };
        let (id, result) = match packet {
            Packet::Ping { id } => (id, Ok(())),
            Packet::Sort { id } => {
                let result: Result<()> = (|| {
                    let mut values = clipboard.read_vector()?;
                    values.sort_by(f64::total_cmp);
                    ui::show_values(&app, &values)
                })();
                (id, result)
            }
            Packet::Shutdown => break,
            _ => continue,
        };
        let _ = communicate.send_packet(MANAGER, &Packet::Finished { id, success:
result.is_ok() }).await;
    }
    ui::quit();
}
```

Лістинг 13 – object3/src/worker.rs

```rust
// SPDX-FileCopyrightText: 2026 Oleksandr Steblevets
// SPDX-License-Identifier: GPL-3.0-only

use crate::App;
use anyhow::Result;
use slint::{ComponentHandle, LogicalPosition};

pub fn configure(app: &App) {
    app.window().set_position(LogicalPosition::new(540.0, 240.0));
}

pub fn show_values(app: &slint::Weak<App>, values: &[f64]) -> Result<()> {
    let display = common::format_values(values);
    app.upgrade_in_event_loop(move |app| app.set_values(display.into()))
        .map_err(|error| anyhow::anyhow!(error.to_string()))?;
    Ok(())
}

pub fn quit() {
    let _ = slint::invoke_from_event_loop(|| { let _ = slint::quit_event_loop(); });
}
```

Лістинг 14 – object3/src/ui.rs

```
// SPDX-FileCopyrightText: 2026 Oleksandr Steblevets
// SPDX-License-Identifier: GPL-3.0-only

import { ScrollView, VerticalBox } from "std-widgets.slint";
export component App inherits Window {
    title: "Object3";
    width: 500px;
    height: 400px;
    in-out property <string> values: "Очікування даних від Lab6...";
    VerticalBox {
        padding: 12px;
        spacing: 8px;
        Text { text: "Відсортований вектор:"; }
        ScrollView { Text { text: root.values; font-family: "monospace"; } }
    }
}
```

Лістинг 15 – object3/src/app.slint

Спільна бібліотека

```rust
// SPDX-FileCopyrightText: 2026 Oleksandr Steblevets
// SPDX-License-Identifier: GPL-3.0-only

pub mod clipboard;
pub mod communicate;
pub mod packets;

use anyhow::Result;
use std::io;

pub fn validate(min: f64, max: f64) -> Result<()> {
    if !min.is_finite() || !max.is_finite() || min > max || !(max - min).is_finite() {
        return Err(io::Error::from(io::ErrorKind::InvalidInput).into());
    }
    Ok(())
}

pub fn format_values(values: &[f64]) -> String {
    values
        .chunks(3)
        .map(|row| {
            row.iter()
                .map(|value| format!("{value:>16.6e}"))
                .collect::<Vec<_>>()
                .join("  ")
        })
        .collect::<Vec<_>>()
        .join("\n")
}
```

Лістинг 16 – common/src/lib.rs

```rust
// SPDX-FileCopyrightText: 2026 Oleksandr Steblevets
// SPDX-License-Identifier: GPL-3.0-only

use crate::packets::Packet;
use interprocess::local_socket::{
    GenericNamespaced, ListenerOptions,
    tokio::{Stream, prelude::*},
};
use std::{io, time::Duration};
use tokio::{
    io::{AsyncBufReadExt, AsyncWriteExt, BufReader},
    sync::mpsc,
    task::JoinHandle,
};

pub struct Communicate {
    packets: mpsc::UnboundedReceiver<Packet>,
    listener: JoinHandle<()>,
}

fn name(role: &str) -> io::Result<interprocess::local_socket::Name<'static>> {
    format!("lab6-{role}").to_ns_name::<GenericNamespaced>()
}

pub fn create(role: &str) -> io::Result<Communicate> {
    let listener = ListenerOptions::new().name(name(role)?).create_tokio()?;
    let (sender, packets) = mpsc::unbounded_channel();
    let listener = tokio::spawn(async move {
        while let Ok(stream) = listener.accept().await {
            let mut stream = BufReader::new(stream);
            let mut message = String::new();
            if let Ok(Ok(size)) = tokio::time::timeout(
                Duration::from_secs(10),
                stream.read_line(&mut message),
            )
            .await
            {
                if size > 0 {
                    if let Ok(packet) = serde_json::from_str(&message) {
                        if sender.send(packet).is_err() {
                            break;
                        }
                    }
                }
            }
        }
    });
    Ok(Communicate { packets, listener })
}

impl Communicate {
    pub async fn send_packet(&self, destination: &str, packet: &Packet) -> io::Result<()> {
        tokio::time::timeout(Duration::from_secs(10), async {
            let mut stream = Stream::connect(name(destination)?).await?;
            let mut message = serde_json::to_vec(packet).map_err(io::Error::other)?;
            message.push(b'\n');
            stream.write_all(&message).await?;
            stream.flush().await
        })
        .await?
    }

    pub async fn receive_packet(&mut self) -> io::Result<Packet> {
        self.packets.recv().await.ok_or_else(|| io::ErrorKind::BrokenPipe.into())
    }

    pub fn has_new_packets(&self) -> bool {
        !self.packets.is_empty()
    }
}

impl Drop for Communicate {
    fn drop(&mut self) {
        self.listener.abort();
    }
}
```

Лістинг 17 – common/src/communicate.rs

```rust
// SPDX-FileCopyrightText: 2026 Oleksandr Steblevets
// SPDX-License-Identifier: GPL-3.0-only

use serde::{Deserialize, Serialize};

pub const MANAGER: &str = "manager";
pub const OBJECT2: &str = "object2";
pub const OBJECT3: &str = "object3";

#[derive(Debug, Serialize, Deserialize)]
pub enum Packet {
    Ping { id: u64 },
    Generate { id: u64, n: usize, min: f64, max: f64 },
    Sort { id: u64 },
    Shutdown,
    Finished { id: u64, success: bool },
}
```

Лістинг 18 – common/src/packets.rs

```rust
// SPDX-FileCopyrightText: 2026 Oleksandr Steblevets
// SPDX-License-Identifier: GPL-3.0-only

use anyhow::Result;

#[derive(Default)]
pub struct Clipboard {
    clipboard: Option<arboard::Clipboard>,
}

impl Clipboard {
    pub fn write_vector(&mut self, values: &[f64]) -> Result<()> {
        if self.clipboard.is_none() {
            self.clipboard = Some(arboard::Clipboard::new()?);
        }
        let text = values.iter().map(ToString::to_string).collect::<Vec<_>>().join("\n");
        self.clipboard.as_mut().unwrap().set_text(text)?;
        Ok(())
    }

    pub fn read_vector(&mut self) -> Result<Vec<f64>> {
        if self.clipboard.is_none() {
            self.clipboard = Some(arboard::Clipboard::new()?);
        }
        let text = self.clipboard.as_mut().unwrap().get_text()?;
        Ok(text.split_whitespace().map(str::parse).collect::<Result<Vec<_>, _>>()?)
    }
}
```

Лістинг 19 – common/src/clipboard.rs

Схема послідовності надсилання та обробки повідомлень

Після введення параметрів Lab6 послідовно надсилає команди об’єктам і очікує підтвердження їх виконання. На схемі наведено успішне виконання завдання.

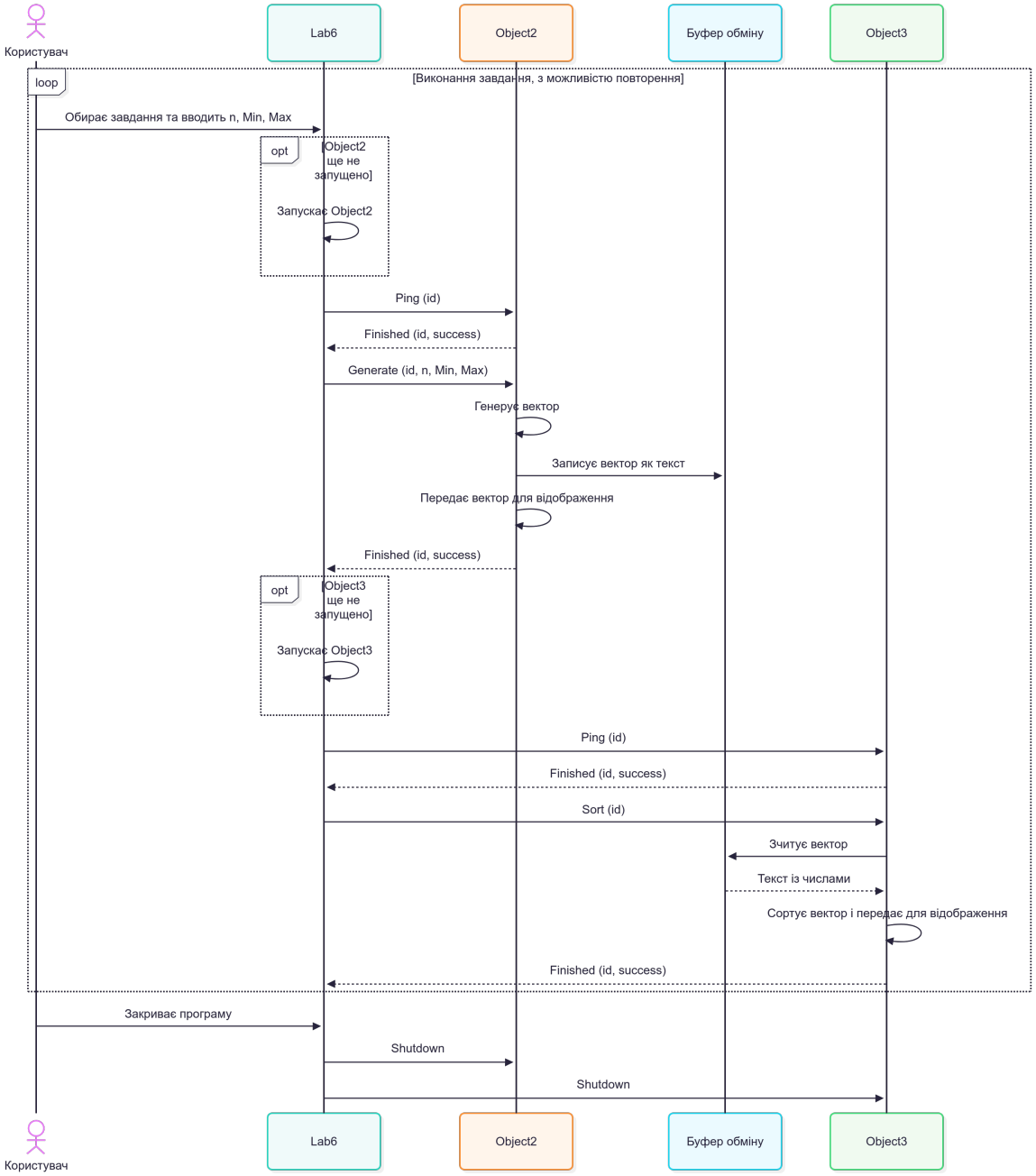


Рисунок 1 – Схема послідовності надсилання та обробки повідомлень

Скріншоти

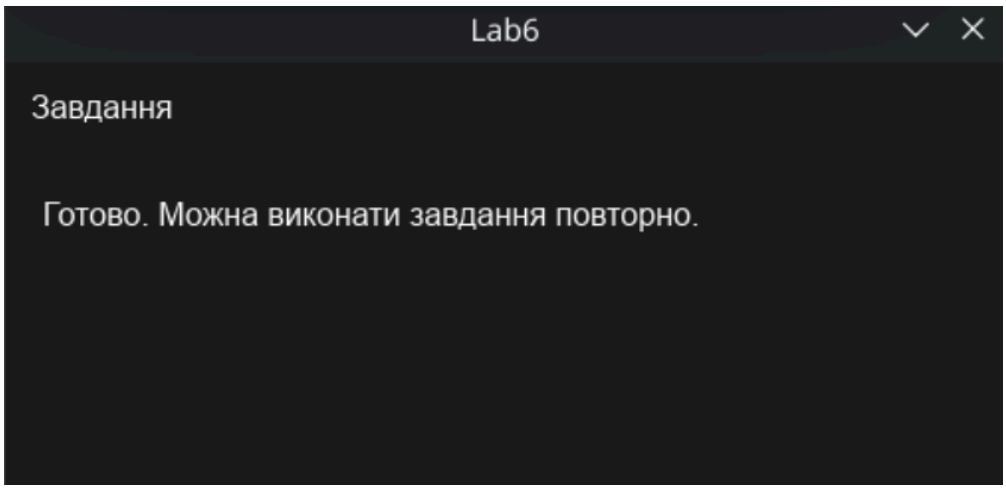


Рисунок 2 – Головне вікно Lab6 після виконання завдання

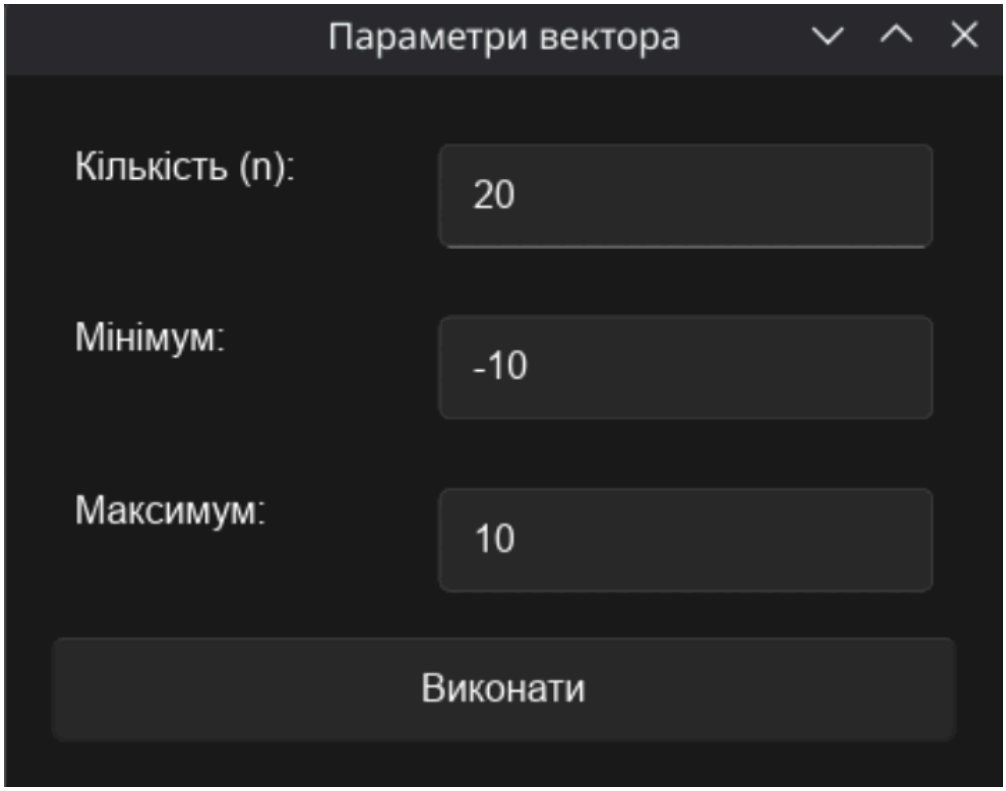


Рисунок 3 – Вікно введення параметрів вектора

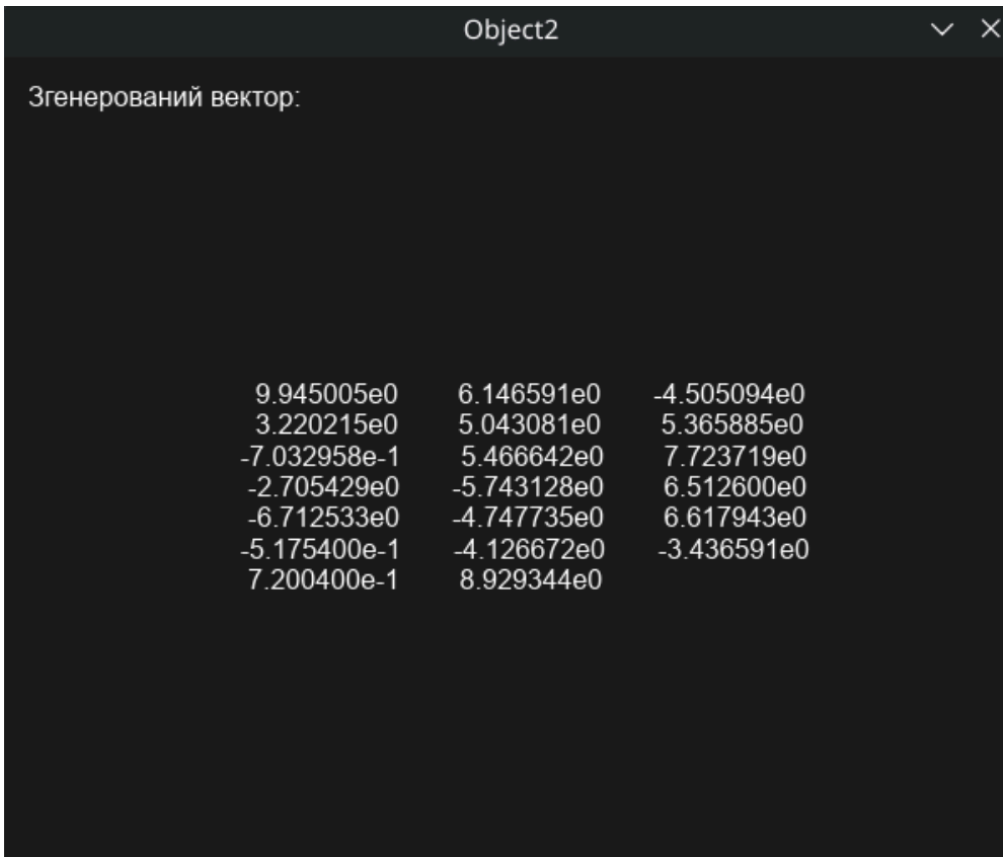


Рисунок 4 – Вікно Object2 зі згенерованим вектором

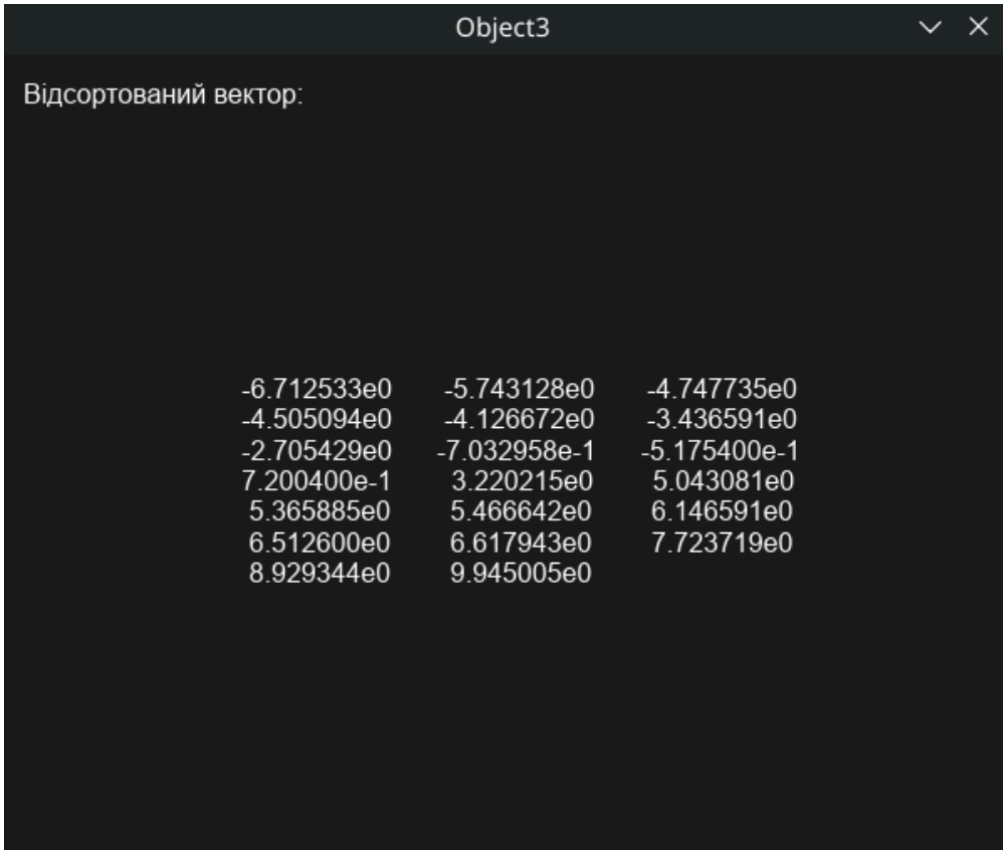


Рисунок 5 – Вікно Object3 з відсортованим вектором

Висновки

Реалізував систему з трьох незалежних програм, керованих повідомленнями. Організував обмін командами через іменовані локальні сокети та передавання вектора через буфер обміну. Отримав практичний досвід взаємодії програмних компонентів, автоматичного запуску програм і послідовної обробки повідомлень.